

Table of Contents

1 Get started	23
1.1 Everything is 0 and 1	23
1.2 Code and data in memory	24
1.3 Functions - a first taste	25
1.4 Developing Programs: VSCode	26
1.5 REPL – write and test code directly	27
1.6 Review questions	27
2 Basic knowledge	30
2.1 A program solves a task - an example	31
2.2 Three types of errors	35
2.3 Naming variables, constants, functions, and files	36
2.4 Operator precedence and associativity	37
2.5 Int and float numerical data types	39
2.5.1 int - integer	39
2.5.2 float - floating point	41
2.5.3 int and float used in the same expression	42
2.5.4 Examine your values - type(), id() and dir()	43
2.5.5 Mutable and immutable data types	44
2.6 str data type - text strings	44
2.6.1 Basic str operations	44
2.6.2 More on escape sequences	47
2.6.3 More on indexing and slicing	48
2.6.4 Overview of useful methods in the str class	49
2.7 The bool data type	52
2.7.1 Truth value for other data types	52
2.8 Conversions between int, float, str, and bool	53
2.8.1 From str to int or float	53
2.8.2 From int or float to str	54
2.8.3 Between int and float	54

2.8.4 Conversion to and from bool	54
2.9 The input() function	55
2.9.1 Common patterns for keyboard input	55
2.10 print() function	57
2.11 Use and misuse of comments	58
2.12 A first look at the math module	59
2.13 A first look at the random module	60
2.14 Formatting text with f-string	62
2.14.1 Decimal precision	62
2.14.2 Comma separators and leading zeros	63
2.14.3 Percent	63
2.14.4 Field width and alignment	63
2.14.5 Order of format specifiers	64
2.14.6 Advanced: Variables instead of constants in the format specifier	64
2.15 Review questions	64
2.16 Exercises	67
3 Bits, Bytes and Number Systems	72
3.1 Introduction	72
3.2 Bits as Code	72
3.3 Bits as Data	73
3.4 Numbers and Characters	74
3.5 The ord() Function	75
3.6 Decimal, binary and hexadecimal number systems	75
3.6.1 Decimal - the base-10 number system	75
3.6.2 Binary - the base-2 number system	75
3.6.3 Hexadecimal – the base 16 number system	75
3.7 Converting between number systems	76
3.7.1 From decimal to binary and hexadecimal - the technique	76
3.7.2 From non-Decimal to decimal - the technique	77
3.7.3 Code for converting from decimal to binary and hexadecimal	77
3.7.4 Code for converting from binary and hexadecimal to decimal	78

3.7.5 Converting between hex and binary	79
3.8 Bitwise Operations	83
3.9 How does the computer calculate?	86
3.9.1 ALU - the CPU's calculator	86
3.9.2 Binary addition	87
3.9.3 Why is subtraction harder?	88
3.9.4 Early attempts at representing negative numbers	89
3.9.5 Two's complement	90
3.10 Review questions	91
3.11 Exercises	93
4 The if - else construct	97
4.1 Simple if - else with relational and logical operators	98
4.1.1 Relational Operators	99
4.1.2 Logical operators	99
4.1.3 Operator precedence	101
4.2 Code Blocks	101
4.3 Code examples with if - else	102
4.3.1 One-way if	103
4.3.2 Two-way if-else	103
4.3.3 Nested if-else	104
4.3.4 elif - flatter and more readable	104
4.3.5 Chained comparisons	105
4.3.6 if - else with combined conditions	105
4.3.7 Shorthand conditional expression (Ternary operator)	106
4.4 The walrus operator :=	107
4.4.1 Guard clauses - exit as early as possible	107
4.5 pass - a syntactic placeholder	109
4.6 Alternative to pass: docstring	109
4.7 The match - case construct	109
4.8 Alternative ways to make choices	111
4.9 Truthiness: True and False in relation to different variable types	113

4.9.1 The value None	114
4.9.2 Truthiness in if statements	115
4.10 Review questions	116
4.11 Exercises	120
5 Lists and the for loop	125
5.1 Introduction	126
5.2 Why do we need lists?	126
5.3 list is a built-in data type	128
5.4 Creating a list and adding elements	129
5.5 Indexing in lists	130
5.6 Operations on sequences	131
5.7 Member methods of the list class	132
5.8 del - deleting elements	133
5.9 Built-in functions for iterables	133
5.9.1 all() and any()	134
5.9.2 map()	134
5.9.3 filter()	134
5.9.4 zip() - combining multiple iterables	135
5.9.5 sorted() and reversed()	135
5.9.6 min(), max() and sum()	136
5.10 A list is a list of references	136
5.11 Copying lists	137
5.11.1 Copying the reference - not a copy!	137
5.11.2 Shallow copy	137
5.11.3 When does shallow copy fall short?	139
5.11.4 Deep copy - complete copy	139
5.12 The for loop - the engine	140
5.12.1 Technique 1: direct iteration over elements	141
5.12.2 Technique 2: iteration with index via range()	142
5.12.3 Technique 3: iteration with enumerate()	142
5.13 The range construct	144

5.14 Copying a list with a for loop.....	145
5.15 Slicing.....	145
5.15.1 General	145
5.15.2 Reversal with slicing - a Python idiom.....	147
5.15.3 Slicing on the left side of an assignment	148
5.16 Multidimensional lists.....	148
5.16.1 Iterating over multidimensional lists	149
5.17 List comprehensions.....	150
5.17.1 Nested comprehensions.....	151
5.17.2 Multidimensional comprehensions.....	151
5.18 join() and lists.....	152
5.19 break and continue.....	153
5.20 Curiosities.....	154
5.20.1 For loop on one line	154
5.20.2 for-else	154
5.21 The while loop	155
5.22 Simulation with random and lists	157
5.23 How fast is our code, really?	159
5.24 Review questions	160
5.25 Exercises	163
6 Functions	169
6.1 Defining a function	170
6.2 Functions without a return value.....	172
6.3 Docstrings	172
6.4 Passing arguments: immutable and mutable	173
6.4.1 Immutable	173
6.4.2 Mutable.....	174
6.5 Naming arguments and parameters	175
6.6 Returning multiple values	175
6.7 Type hints.....	176
6.7.1 Type hints are not enforced	177

6.7.2 Type hints for lists.....	178
6.7.3 Values that can be None.....	178
6.7.4 Several possible types	179
6.7.5 A brief note on generics and type	180
6.8 Keyword arguments.....	181
6.9 Default values	181
6.10 *args and **kwargs.....	182
6.10.1 *args - positional arguments.....	183
6.10.2 **kwargs- keyword arguments	184
6.10.3 *args and **kwargs combined.....	185
6.11 Command-line arguments.....	185
6.12 Variable scope.....	187
6.12.1 Scope and namespace	187
6.12.2 Loops and conditionals do not create their own scope	188
6.12.3 Functions create local scope	189
6.12.4 Comprehensions create local scope.....	189
6.12.5 Explicit global declaration.....	189
6.12.6 Shadow variables.....	190
6.13 Importing functions from other modules	190
6.14 A case: lottery draw.....	192
6.15 Review questions	196
6.16 Exercises	197
7 Tuples, dictionaries and sets	206
7.1 Introduction.....	207
7.2 Tuple	207
7.2.1 Motivation and use cases.....	207
7.2.2 Creating and using tuples.....	208
7.2.3 Tuple unpacking	210
7.3 Dictionary.....	212
7.3.1 Motivation and use cases.....	212
7.3.2 Basic operations.....	212

7.3.3 Various useful methods.....	214
7.3.4 Iterating over a dictionary - dictionary views	215
7.3.5 Sorting a dictionary	215
7.3.6 Key requirements – hashability.....	216
7.3.7 Example: counting occurrences.....	217
7.3.8 Example: nested dictionaries	218
7.3.9 Constructor calls as values	222
7.4 Sets.....	224
7.4.1 Motivation and use cases.....	224
7.4.2 Creating a set.....	224
7.4.3 Basic operations.....	225
7.4.4 Set operations	225
7.4.5 Example: counting Python keywords.....	226
7.5 collections.Counter - counting occurrences.....	229
7.5.1 A practical example - counting words in a text:	230
7.6 Type hints for tuple, dict and set	230
7.6.1 Tuple.....	230
7.6.2 Dict	231
7.6.3 Set	233
7.7 Review questions.....	233
7.8 Exercises	234
8 Classes and objects - the basics	245
8.1 Introduction	245
8.2 Creating a class	246
8.3 <code>__init__()</code> - the constructor	247
8.4 What is self?.....	248
8.5 Dot notation:	248
8.6 When should something become a class?.....	249
8.7 A more advanced class: Account.....	249
8.7.1 Basic Account class	250
8.8 Python has no private data members.....	252

8.9 Account with transaction log	253
8.10 Properties for access control.....	255
8.10.1 Another property example: Temperature	258
8.11 dataclass - when the class is just data	260
8.11.1 Default values for mutable fields.....	261
8.12 Class methods and class variables	263
8.13 Operator overloading	267
8.13.1 Overloading indexing and the in operator	269
8.14 Overloading vs overriding	271
8.15 Truthiness of objects.....	271
8.16 Review questions	273
8.17 Exercises	274
9 Unit testing.....	283
9.1 Introduction	283
9.2 assert - simple testing in the REPL.....	284
9.3 pytest - structured testing.....	284
9.4 Naming conventions	287
9.5 Code and test files.....	287
9.6 Assert expressions in pytest.....	287
9.7 Testing approximate values	288
9.8 Testing many values - @pytest.mark.parametrize	289
9.9 Fixtures - the same starting point for each test.....	290
9.10 Fixture scope - initialising once for the whole file.....	292
9.11 Guidelines for good unit tests	293
9.12 Test coverage.....	294
9.13 AI and unit testing.....	295
9.14 Review questions	295
9.15 Exercises	296
10 More on classes and objects	302
10.1 Introduction	302
10.2 UML- class diagrams	303

10.3 The is-a relation: inheritance	304
10.4 The has-a relation: composition and aggregation	306
10.4.1 Composition	306
10.4.2 Aggregate	308
10.5 super() - calling the superclass	309
10.6 Polymorphism	310
10.7 isinstance()	311
10.8 Abstract classes and the abc module	312
10.9 Account extended into a class hierarchy	314
10.10 object - the ultimate superclass	318
10.10.1 __repr__ and eval()	320
10.10.2 __eq__() and __hash__() - two sides of the same coin.....	321
10.11 Multiple inheritance	324
10.12 The Iterator protocol.....	325
10.13 collections.abc.Iterator	330
10.14 Generators – iterators without a class	331
10.14.1 Generator expressions	332
10.15 Review questions.....	333
10.16 Exercises.....	335
11 File handling and exceptions	346
11.1 Introduction	347
11.2 Text files: opening and writing	347
11.2.1 write()	347
11.2.2 File modes.....	348
11.2.3 writelines().....	348
11.2.4 Append mode	349
11.3 Text file: reading	349
11.3.1 read()	349
readline()	350
11.3.2 readlines()	350
11.3.3 Iterating over the file object	351

11.4 About the file object.....	351
11.5 Text coding.....	352
11.6 Exception handling.....	352
11.6.1 Specific exception handling.....	353
11.6.2 The exception-object.....	353
11.6.3 The exception hierarchy.....	354
11.6.4 else and finally	355
11.6.5 Creating our own exceptions	356
11.6.6 Custom exceptions for Account.....	357
11.7 Context managers and with-as	362
11.8 pathlib - modern file path handling.....	363
11.8.1 Concepts related to paths	363
11.8.2 Useful methods and properties.....	364
11.8.3 Pathlib example	364
11.8.4 read_text() and write_text()	366
11.9 Serialisation.....	367
11.9.1 pickle.....	367
11.9.2 json.....	372
11.10 CSV-files.....	377
11.10.1 Reading and writing with reader and writer.....	378
11.10.2 DictReader and DictWriter.....	379
11.10.3 Delimiters.....	381
11.10.4 pickle, json or CSV?	381
11.11 datetime and timedelta.....	381
11.11.1 Creating datetime objects.....	382
11.11.2 Parsing from text and formatting to text.....	382
11.11.3 timedelta - durations.....	382
11.11.4 Arithmetic with datetime and timedelta	383
11.11.5 A real-world example	383
11.12 Review questions.....	384
11.13 Exercises.....	386

12 GUI programming with Tkinter	397
12.1 Introduction	398
12.2 What is event-driven programming?	399
12.3 The first GUI program with Tkinter.....	399
12.4 Tkinter and the event queue.....	401
12.5 Basic Tkinter GUI elements.....	402
12.5.1 The main window	402
12.5.2 Widgets	402
12.6 The usefulness of keyword arguments.....	406
12.7 The config()-method.....	406
12.7.1 The alternative to config() - indexing with the option dictionary	408
12.8 Structuring a GUI with Frame and geometry managers	409
12.8.1 Geometry-managers	409
12.9 More about event handling	410
12.9.1 command without parameters.....	410
12.9.2 command with parameters - lambda	410
12.9.3 bind() for connecting event listeners	412
12.9.4 bind + parameter passing with lambda.....	414
12.9.5 The event object	416
12.10 Listbox, scrollbar, text and canvas	417
12.10.1 Listbox	417
12.10.2 Scrollbar	421
12.11 Tkinter variables - StringVar, IntVar and DoubleVar.....	424
12.12 Canvas	429
12.12.1 Canvas and mouse events	429
12.12.2 The itemconfig() method.....	435
12.13 Animations	437
12.13.1 Simple ball animation.....	438
12.13.2 A slightly more sophisticated ball animation	440
12.14 Dialog boxes.....	442
12.15 Review questions.....	446

12.16 Exercises.....	447
13 Advanced functions: closures, decorators and more	457
13.1 Introduction	457
13.2 Closures - functions that remember.....	458
13.3 Lambda as a closure.....	460
13.3.1 Lambda - summary of the syntax.....	461
13.3.2 Factory functions	462
13.4 Decorators - extend functions without changing them	463
13.4.1 The @-syntax.....	464
13.4.2 Case: measuring execution time of sorting algorithms.....	465
13.5 @property and the property class	468
13.5.1 Code example: creating and using a property object.....	468
13.5.2 Code example: using the @property decorator	469
13.5.3 Creating read-only attributes.....	470
13.6 nonlocal.....	471
13.7 Review questions	472
13.8 Exercises	473
14 Recursion	479
14.1 Introduction	479
14.2 Recursion can seem difficult.....	480
14.3 A simple example	480
14.4 Recursion uses the stack (iteration does not).....	481
14.5 Concepts related to recursion	482
14.5.1 Linear recursion - one call	482
14.5.2 Tree recursion - multiple calls.....	482
14.5.3 Tail recursion.....	483
14.5.4 Memoisation	484
14.6 Recursion and mathematical induction	484
14.6.1 The similarity demonstrated through the summation formula	484
14.7 Tips and advice.....	485
14.8 Computing factorial.....	485

14.9 Fibonacci numbers	486
14.9.1 Naive implementation.....	486
14.9.2 Improved Fibonacci (1) - recursive with memoisation.....	487
14.9.3 Improved Fibonacci (2) - iterative variant.....	488
14.10 Recursive traversal of directory/file structure	488
14.11 Tower of Hanoi	490
14.11.1 The problem.....	490
14.11.2 Why recursion fits perfectly here	490
14.11.3 Step 2 is not the base case	491
14.11.4 Walkthrough for 3 disks, step by step.....	491
14.11.5 The recursive solution.....	491
14.12 When should recursion not be used?	494
14.13 Review questions.....	495
14.14 Exercises.....	496
15 Appendix: Regular expressions.....	504
15.1 What are regular expressions?	504
15.2 Before regex: built-in string methods.....	504
15.3 The re module.....	505
15.4 Metacharacters.....	506
15.5 Metasequences.....	507
15.6 Code example - basic usage.....	508
15.7 Example - validating and finding hex colour codes.....	513
15.8 Example - validating email addresses.....	515
16 Appendix: Operator precedence and associativity	519
17 Appendix: Downloading and installing Python.....	522
17.1 Windows	522
17.2 Mac	522
17.3 Linux.....	523
17.4 Verifying the installation.....	523
17.5 REPL - Python directly in the terminal.....	524
18 Appendix: Downloading and installing VS Code.....	526

18.1 Download and installation	526
18.1.1 Windows.....	526
18.1.2 Mac.....	526
18.1.3 Linux	527
18.2 First startup and folder structure.....	527
18.3 Installing the Python extension.....	528
18.4 Verifying the setup.....	528
19 Appendix: Git and GitHub.....	530
19.1 What is a repository?	530
19.2 Prerequisites	530
19.3 Creating an SSH key.....	530
19.4 Your first Git project.....	532
19.4.1 Step 1: Create folder and open in VS Code	532
19.4.2 Step 2: Initialise a local repository	532
19.4.3 Step 3: Add files and commit.....	532
19.4.4 Step 4: Connect to GitHub and push.....	533
19.5 Cloning an existing repository	533
19.6 .gitignore	533
19.7 Important Git commands.....	534
19.8 Branches	534
19.9 Further reading.....	535
20 Appendix: Debugging in VS Code	536
20.1 Why debug?	536
20.2 Running in debug mode.....	537
20.3 Breakpoints.....	537
20.4 The Variables panel - see what is happening.....	538
20.5 Variables that "come and go"	539
20.6 Navigating through the code	539
20.7 Call Stack.....	540
20.8 Watch - following a variable	541
20.9 An example - finding a bug.....	542